網路應用軟體設計 2026 | 期末專題

多人幾 A 幾 B 網路對戰系統

BC/1.0 協定實作：修改後的系統架構
AI 代理人改由外部 LLM 決定喊數，演算法作為備援



4A0B

被回報 4A0B 的玩家出局

伺服器・人類客戶端・ AI 代理人 | Python 3.10+ 標準函式庫 | Windows ／ WSL ／ Linux

專題需求與達成情況

1 多人對戰

2–6 人同時玩幾 A 幾 B ；一次喊數對所有存活對手有效，結果全部公開

2 BC/1.0 協定

TCP 上一行一則 UTF-8 JSON ；客戶端 9 種、伺服器 14 種訊息，可跨組互通

3 三種角色

伺服器、人類客戶端、 AI 代理人，全在同一支 bc.py ，以子命令切換

4 公平機制

開局只送 SHA-256 承諾值（加鹽），揭示時驗證，說謊者取消資格

5 AI 接入 LLM

由外部 LLM 決定喊數；逾時或答案不合格就退回演算法

6 可驗證

只用標準函式庫； selftest 48/48 、 bc_test 72/72 、 LLM 測試 33/33 全過

遊戲規則與一局的五個階段

1 大廳

`HELLO → WELCOME`
`LOBBY`

分配 p1、p2…，人數到齊或輸入 start 開局

2 承諾

`GAME_START`
`COMMIT → COMMITS`

每人只送 SHA-256 承諾值，逾時移出

3 回合

`TURN → GUESS`
`GUESS_MADE`
`FEEDBACK`
`TURN_RESULT`

喊數對所有存活者有效，結果全部公開

4 揭示

`REVEAL_REQUEST`
`REVEAL → REVEALED`

有人 4A0B 立即要求揭示並驗證

5 終局

`GAME_OVER`

只剩 1 人或達回合上限，公布排名

5040 種秘密

每人一個 4 位、數字互不重複的秘密（10×9×8×7）

4A0B 出局

被回報 4A0B 的玩家出局；只剩 1 人或達回合上限即終局

名次規則

存活者 > 被猜中者（越晚出局越前）> 棄權者； DQ 沒有名次

BC/1.0 訊息與公平性要求

客戶端 → 伺服器（9 種）

HELLO | COMMIT | GUESS | FEEDBACK | REVEAL | CHAT | PING

PONG | BYE

伺服器 → 客戶端（14 種）

WELCOME | LOBBY | GAME_START | COMMITS | TURN | GUESS_MADE | TURN_RESULT

REVEAL_REQUEST | REVEALED | GAME_OVER | CHAT | PING | PONG | ERROR

1 承諾 – 揭示

開局只送 SHA256(secret + ":" + salt)，salt 16 bytes；伺服器始終不知道秘密

2 一致性檢查

依公開回報維護候選集合，變空 = 說謊，判 INCONSISTENT 取消資格

3 LLM 不影響公平

LLM 看不到自己的 secret 與 salt；回報與揭示由本機誠實計算

收到 TCP 資料後，伺服器怎麼判斷

1 接受連線

超過 64 條直接關閉；10 秒內沒送 HELLO 就回 TIMEOUT 並斷線

→

2 切出一行

read(65536) 累積到緩衝，找到 LF 才切一行，去掉 CR；超過 4096 bytes 回 LINE_TOO_LONG 並斷線

→

3 解析 JSON

解析失敗、巢狀過深、NaN 回 BAD_JSON；必須是含 type 字串的物件

→

4 檢查類型

HELLO 之前只收 HELLO（否則 UNEXPECTED 並斷線）；不認得的 type 回 UNKNOWN_TYPE

→

5 分派處理

呼叫 on_<TYPE>()，檢查欄位型別、目前階段與值；任何例外只關閉這條連線

例：收到 GUESS 時依序檢查，第一個不符合的就回對應錯誤碼

turn_id 是整數	現在是回合中	turn_id 是本回合	輪到你喊	還沒喊過	4 位不重複
✗ BAD_FIELD	✗ UNEXPECTED	✗ STALE_TURN	✗ NOT_YOUR_TURN	✗ UNEXPECTED	✗ INVALID_VALUE

asyncio 併發與每回合的判定

併發模型： asyncio 單執行緒

- 每條連線一個讀取協程，遊戲流程另一個協程
- 流程協程用 asyncio.Event 加逾時等待「收齊或時間到」
- 訊息處理函式是同步的：同一封包內多則訊息一定依序處理，沒有競態
- 對方不讀資料、待送緩衝超過 1 MiB 就斷線
- 時限都可用命令列調整，可設 1.5 秒這類小數

一回合的判定順序



指定喊數者，等 GUESS （guess_timeout）
逾時跳過；連續 max_skips 次判棄權

等所有被猜者 FEEDBACK （feedback_timeout）
沒回報或斷線判棄權

依回報過濾每人的候選集合
集合變空判 DQ ； 4A 判出局

廣播結果、出局、棄權、 DQ 名單
有人出局就送 REVEAL_REQUEST

分工：程式負責精確計算，LLM 負責策略取捨

本機程式（精確、可驗證）

- 秘密產生、COMMIT 承諾值
- FEEDBACK 的幾 A 幾 B、REVEAL
- 驗證別人的揭示
- 每位對手的候選集合
- 每個喊數的分組數 parts、最壞情況 worst
- 輸出格式與品質檢查、時限控制

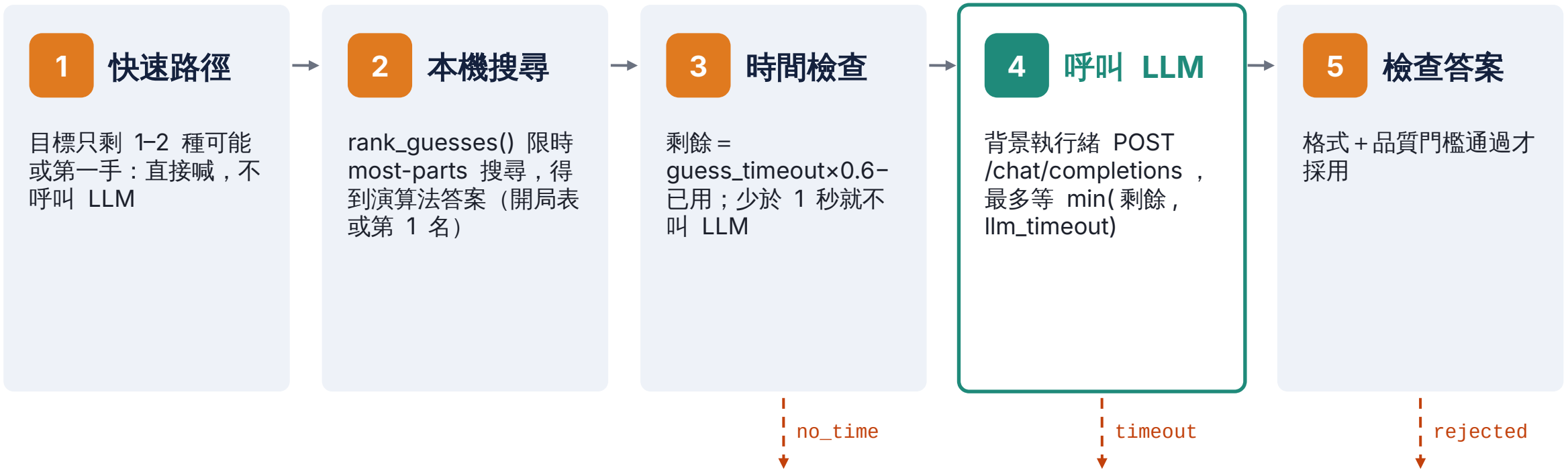
局面 JSON →

← `{guess, reason}`

外部 LLM （策略選擇）

- 在程式算好的選項中挑一個喊數
- 權衡「鎖定目標」與「同時縮小多位對手」
- 判斷何時該直接賭候選答案
- 附一句 30 字內的理由，顯示在觀戰畫面

每回合的決策流程（AI.choose）



任何一步失敗 → 本回合改喊演算法答案（記為 fallback: 原因），下回合再試 LLM
每局結束印出「喊數來源統計」，例如：llm 7、fallback:timeout 1。時限 1.5 秒的測試對局會自動全用演算法，不會因 LLM 而逾時被跳過。

三種常見用法：照格子順序按 Shift+Enter

第一次使用

- `0` 載入共用設定
- `1` 環境檢查： LLM 連得上嗎、有哪些模型
- `2-1` 看目前用哪個 profile
- `2-3` 試問 LLM 一次，看選了什麼、花多久

看一局 LLM 對戰

- `3-1` 啟動伺服器（設人數、時限）
- `3-2` 加入 LLM-AI 與 ALGO-AI
- `3-4` 即時觀戰：每手喊數、回報、LLM 理由
- `3-6` 看名次； 3-7 全部停止

比較 LLM 與演算法

- `4-1` 設定局數 N 、是否邊打邊看
- `4-2` 繼續觀戰，打完統計勝場
- `3-6` 看每局喊數來源統計（ llm ／ fallback ）

換模型：在 2-2 改 active 或 model 後執行即存檔，新啟動的 AI 就套用。 **WSL 連不到 Windows 上的 LLM ：**第 1 格會自動找 Windows 主機 IP 並告訴你怎麼改。

05 結語

協定不變，LLM 隨插即用

需求全數達成，三套測試全過。 LLM 經 OpenAI 相容 API 接入 AI 客戶端，改 llm_config.json 就能換模型； MAIN.ipynb 一本完成啟動、觀戰、對戰與測試。

下一步

- 用 MAIN.ipynb 第 4 節跑 LLM 對演算法至少 100 局，統計勝率與喊數來源
- 比較不同模型與提供者的延遲和勝率
- 跨組對戰
- 完成期末設計文件